Peer-Isolated Recommendation: When Content-Only Personalization Beats Collaborative Signal, and When It Doesn't

Bidit Das
Independent Researcher | Dallas, TX
biditdas18@gmail.com

August 31, 2026

Abstract

We present **Essence**, a personal embedding cluster framework for recommendation that operates exclusively within a single user's engagement history, without consulting cross-user interaction data. We evaluate Essence on three datasets spanning distinct domains — Last.fm-1K (music), Amazon Books (e-commerce), and MovieLens-25M (film) — against a real collaborative-filtering baseline, a non-personalized popularity baseline, a content-based average-embedding baseline, three recency-based baselines (last-item, uniform-average, and exponentially-weighted retrieval), and two hand-implemented multi-interest neural baselines (MIND [1], ComiRec [2]).

Essence's performance relative to these baselines is **domain-dependent**: across the three evaluated datasets, Essence's rank among the ten evaluated systems is associated with the performance of the evaluated collaborative baselines in the domain; establishing this as predictive would require additional datasets and an independently-defined domain property, not one measured post hoc via baseline performance. This domain-dependence is not unique to Essence: Recency-Weighted, a non-clustering baseline using identical embeddings, shows the identical win/loss pattern against the same baselines on Recall@10 (Section 5.2), indicating the property belongs to the broader no-cross-user-pooling architecture family rather than to K-means clustering specifically. K (the number of interest clusters) is selected per dataset on a validation split carved from training data, not fixed in advance — $K = 10$ for Last.fm-1K and Amazon Books, $K = 15$ for MovieLens-25M. On Amazon Books, where collaborative and popularity baselines are structurally weak (Recall@10 of 0.0031 and 0.0021 respectively), Essence ranks 3rd of 10 on Recall@10 (0.0253): it loses only to Last-Item (0.0311) and is not significantly different from Recency-Weighted (0.0260, $d = -0.009$) and Avg-Last-10 (0.0235, which it now numerically beats). It significantly beats collaborative filtering, popularity, content-averaging, and both neural multi-interest baselines on Recall@10 (d ranges 0.099–0.282), and significantly beats Random, Popularity, and MIND on LT-Recall@10 specifically; it is not significantly different from CF-ItemKNN, Content, ComiRec, Last-Item, Recency-Weighted, and Avg-Last-10 on LT-Recall@10. On Last.fm-1K, Essence ranks 2nd of 10 on both Recall@10 (0.0281) and LT-Recall@10 (0.0142); it is not significantly different from any of the three recency-only baselines (numerically ahead of Last-Item and Avg-Last-10, behind Recency-Weighted, 0.0304, $d = -0.127$, not significant). On MovieLens-25M, where the evaluated collaborative baselines perform strongly (item-based CF reaches 0.0850 Recall@10, driven almost entirely by the platform's densely-rated popular titles), Essence ranks 8th of 10 systems, losing to collaborative filtering, popularity, and both neural baselines by the largest effect sizes observed in this study (Cohen's d down to -0.55). A silhouette-stratification test, comparing Essence against Recency-Weighted within per-user clustering-quality tertiles, finds that K-means clustering is not empirically supported as the source of Essence's advantage where an advantage exists: no stratum, on any dataset or metric, shows Essence significantly ahead of Recency-Weighted, though only one of sixteen clean such comparisons (MovieLens-25M's highest-silhouette stratum on LT-Recall@10) now shows Essence significantly *behind* it either, down from six of seventeen before validation-selecting K. The question this paper does not fully resolve — how much of Essence's advantage, where it exists, comes from

clustering specifically versus recency-weighted retrieval alone — remains open.

We report all primary baseline and stratification comparisons with paired bootstrap significance testing (10,000 resamples), Benjamini–Hochberg correction for multiple comparisons, standardized effect sizes, and BCa robustness cross-checks — replacing an earlier confidence-interval-overlap methodology that does not test the quantity of interest. We also report a popularity-decile breakdown showing that Essence's long-tail advantage, where present, is concentrated in moderately-rare-but-observed items rather than true cold-start items, and an artist/author-identity shortcut analysis showing that a disproportionate share of Essence's hits come from same-artist/author candidates despite these being a minority of its recommendations. We frame Essence not as a universally superior candidate generator, but as a data point on the domain conditions under which peer-isolated, content-only personalization is or isn't the right architectural choice. Code and data: https://github.com/biditdas18/essence

1 Introduction

Recommendation systems built on collaborative filtering (CF) derive their signal from cross-user interaction patterns: a user's future interactions are predicted from the behavior of users who interacted similarly in the past. This approach performs well on standard benchmarks, particularly on metrics dominated by popular items, but it carries a structural property worth examining directly: items with sparse interaction histories are, by construction, underrepresented in any signal derived from cross-user co-occurrence. A user with a narrow, idiosyncratic taste profile (engaging consistently with low-interaction items across a domain) receives recommendations shaped by aggregate similarity to other users, not by the specific combination of properties their own history reflects.

This work originated from a practical observation in music recommendation: users with precise, narrow taste profiles are reliably steered toward broadly popular artists that are only loosely similar, while artists matching the specific texture of that taste remain unsurfaced regardless of how long the user searches the catalog manually. This pattern motivated two questions addressed here. This paper is structured as an evaluation study, not a method-introduction paper: the central question is whether a specific architectural choice — K-means interest clustering — provides measurable benefit over a simpler recency-based alternative, tested under conditions designed to give that choice its best chance to matter. First, the original motivating question: can a recommendation signal be constructed using only a single user's own history, without any cross-user interaction data, and if so, does it recover items that collaborative methods structurally underrepresent? Second, a question that only becomes answerable once the same comparison is run across multiple, structurally different domains: under what domain conditions does this approach help versus hurt, and is that pattern associated with properties of the domain rather than discovered post hoc per dataset? A one- or two-dataset evaluation cannot distinguish a domain-general advantage from a domain-specific artifact; the three-dataset evaluation reported here (Section 5) is designed specifically to make that distinction possible.

We propose Essence, a framework that embeds a user's consumed items into a semantic space using a pre-trained sentence encoder, clusters them into K interest profiles via K-means, and generates recommendations by retrieving items similar to the centroid closest to the user's recent activity. We refer to this as the *peer-isolation property*: no cross-user interaction graph is consulted at any stage. This is a deliberate architectural constraint, not an oversight: it is the mechanism under test. We describe the resulting objective as depth-optimized: the goal is to retrieve items deep in a user's long-tail interest space, rather than to maximize aggregate engagement on broadly popular items.

The contributions of this work are:

1. A **three-dataset evaluation** testing whether K-means interest clustering provides mea-

surable benefit over a simpler recency-based alternative (Section 5), yielding a **domain-dependence finding** across three datasets spanning music, e-commerce, and film: Essence's relative standing among ten evaluated systems is associated with the performance of the evaluated collaborative baselines in the domain (establishing this as predictive would require additional datasets and an independently-defined domain property), evidenced by effect sizes ranging from Essence's strongest win (Last.fm-1K, beating Random with $d = +0.301$) to its strongest loss (MovieLens-25M, losing to CF-ItemKNN with $d = -0.550$) — the losses are consistently the larger-magnitude effects throughout this evaluation. This domain-dependence pattern is shown to belong to the broader no-cross-user-pooling architecture family rather than to K-means clustering specifically, since Recency-Weighted alone reproduces it on Recall@10 (Section 5.2).

2. The **Essence architecture** itself: a personal recommendation framework requiring no collaborative signal and no supervised or cross-user recommender training, operating entirely on a single user's history.

3. A **recency-based active-cluster-selection** mechanism: an ablation against a static mean-embedding alternative (Section 5.8) supports retaining it, while a separate comparison against three recency-only baselines that do not cluster at all (last-item, uniform-average, exponentially-weighted retrieval) — absent from the original evaluation — finds Essence rarely significantly ahead of them, evidence against clustering's distinct value rather than for it (Section 5.10).

4. **Two hand-implemented, from-scratch multi-interest baselines** (MIND, following Li et al. [1]; ComiRec, following Cen et al. [2]) — neither is available in standard recommendation libraries, and neither original paper's benchmark numbers are directly comparable under this paper's evaluation protocol (Section 4), so correctness is verified against each model's architectural specification rather than against a published number (Section 4 describes the verification methodology and a bug it caught in our own implementation).

2 Related Work

2.1 Collaborative Filtering

Matrix factorisation methods [5] and neural extensions [4] remain dominant in production recommendation. Their principal limitation is popularity bias: items with sparse interactions are poorly represented, resulting in near-zero recall for long-tail items. CF is optimised for the median user: it performs well on what is broadly consumed and comparatively weakly on what is individually resonant.

2.2 Multi-Interest Retrieval

MIND [1] introduced dynamic routing to extract multiple interest capsules from a user's interaction history, explicitly addressing the failure of a single user vector to represent diverse tastes. ComiRec [2] extended this with explicit diversity controls at serving time. Later architectures relax MIND's routing mechanism differently: SINE infers a sparse, adaptively-selected concept set per user rather than a fixed capsule count [17], while Re4 adds contrastive and reconstruction objectives to make individual interest embeddings more distinguishable and more faithful to the items that justify them [18]. Both remain in the same data-level lineage as MIND and ComiRec — cross-user interaction data trains the routing/attention mechanism itself, the distinction Essence's comparison to MIND/ComiRec (below) is actually about.

Essence shares the multi-interest intuition with MIND and ComiRec, the recognition that a user's preferences cannot be collapsed into a single vector. However, the similarity ends at the surface. The distinction operates at two levels.

Data level. MIND and ComiRec require population-level interaction graphs. Their dynamic routing is trained on millions of (user, item, interaction) triples across the full user base. Cross-user behavioural data is not optional: it is the mechanism by which interest capsules are learned. Essence requires no data from any other user, ever. The K-means centroids are derived entirely from one user's own history.

Representation level. Because MIND's item embeddings are trained on co-occurrence patterns across users, what an item *means* in that space is partly defined by who else interacted with it. An obscure track heard by very few users is underrepresented or invisible in that embedding space, as its co-occurrence signal is too sparse to be learned reliably. Essence uses a pre-trained sentence encoder on item metadata only. The embedding captures metadata-derived semantic information about the item, independent of how many people have interacted with it. A singleton item can be embedded as readily as a chart-topping one; the quality of that representation depends on how informative the item's metadata is, not on its interaction count.

This distinction suggests a hypothesis: Essence should recover long-tail items at higher rates than systems whose embeddings are shaped by co-occurrence sparsity. We test this directly against hand-implemented MIND and ComiRec baselines (Section 4), and the result is more qualified than the hypothesis alone predicts: Essence significantly outperforms both on Amazon Books, but on MovieLens-25M both neural baselines substantially outperform Essence instead, by some of the largest effect sizes observed in this study (Section 5.2). The data-level and representation-level distinctions described above are architectural facts regardless of outcome; whether they translate into a long-tail recovery advantage in practice depends on domain conditions, not on the architectural distinction alone.

Table 1 summarises the architectural distinctions between Essence, MIND, ComiRec, and standard CF.

Table 1: Architectural comparison: Essence vs. prior systems. The primary distinction is not the clustering mechanism, it is the data requirement and what each system is structurally able to score. "Distinguishes among zero-interaction items": CF, MIND, and ComiRec derive representations from co-occurrence and so cannot distinguish among items with zero training interactions, since such items receive identical (typically zero) co-occurrence scores; Essence's metadata-derived embeddings differ per item regardless of interaction count. All four systems are empirically evaluated in this work (Section 5), which reports the domain-dependent long-tail recovery result in full.

Property	**CF**	**MIND**	**ComiRec**	**Essence**
Cross-user data required	Yes	Yes	Yes	**No**
Multiple interest profiles	No	Yes	Yes	**Yes**
Gradient-based training required	No	Yes	Yes	**No**
Distinguishes among zero-interaction items	No	No	No	**Yes**
Embedding source	Co-occurrence	Co-occurrence	Co-occurrence	**Item metadata**

2.3 Session-Based and Content-Based Methods

BERT4Rec [3] and GRU4Rec [6] model sequential user behaviour using transformer and recurrent architectures, capturing short-term context effectively. Content-based filtering [15] relies on item feature similarity without interaction logs but typically averages preferences into a single vector, losing intra-user diversity. Essence's retrieval mechanism belongs to this lineage — items are represented by pre-trained text embeddings over metadata, not by collaborative signal — but adds a per-user K-means decomposition that classical content-based filtering does not.

Essence occupies a complementary position: it captures intra-user diversity (like multi-interest methods) without cross-user interaction data (like content-based methods), using simple interpretable clustering rather than a trained sequential model. We acknowledge that BERT4Rec offers richer sequential modelling in high-interaction settings; Essence targets privacy-first, low-infrastructure deployment contexts where such models are not viable.

2.4 Reproducibility and Progress in Recommender Systems Research

Ferrari Dacrema et al. [12] showed that a majority of neural recommendation papers at top venues failed to outperform simple, well-tuned non-neural baselines when those baselines were given a fair tuning budget, and that a substantial fraction of claimed improvements did not reproduce under careful re-evaluation. A follow-up analysis [13] extended this finding across a broader set of papers and venues, identifying weak baselines, inconsistent evaluation protocols, and selective reporting as recurring, systemic causes of inflated claims of progress in the field. This paper's own findings are consistent with, and extend, that line of critique to a different architecture family: content-only, no-cross-user-pooling personalization. Essence's headline long-tail advantage survives paired bootstrap significance testing on Amazon Books but is directional and not statistically significant on the smaller Last.fm-1K, and on MovieLens-25M — a domain with strong collaborative signal — Essence loses to collaborative filtering, popularity, and both neural multi-interest baselines by the largest effect sizes observed anywhere in this study (Section 5.2). This is the discipline Ferrari Dacrema et al. call for: a fair, statistically grounded comparison against strong, simple baselines, with mixed results reported alongside favorable ones.

3 The Essence Framework

3.1 Notation

Table 2 defines all mathematical notation used in this section. For each symbol a plain-English equivalent is provided.

Table 2: Notation Legend

Symbol	Plain-English Meaning
H_u	All items user u has consumed (full history)
$i_1, i_2, \ldots$	Individual items in the user's history, in order
$\phi(i)$	Embedding of item i, a 384-dim vector describing its content
E_u	All embeddings stacked into a matrix, one row per item
K	Number of interest clusters (validation-selected per dataset: $K = 10$ Last.fm-1K/Amazon Books, $K = 15$ MovieLens-25M)
$c_1, \ldots, c_K$	The K cluster centres, one per interest profile
c_k^*	Active cluster centre, closest to the user's recent activity
$\mu(\cdot)$	Mean (average) of a set of vectors
$\cos(\mathbf{a}, \mathbf{b})$	Cosine similarity: how alike two vectors are
$I \setminus H_u$	All items the user has *not* yet consumed, the candidate pool
R_u	Final ranked recommendation list for user u
M	Number of recommendations to return (we use $M = 10$)

3.2 Problem Formulation

Given a user's history H_u (all items they have consumed), the goal is to produce a ranked list $R_u \subseteq I \setminus H_u$ of items they have not yet consumed but are likely to enjoy. Essence accomplishes this using only H_u and a shared embedding function $\phi : I \to \mathbb{R}^d$. No data from any other user is used at any stage.

3.3 Item Embedding

Each item is described by a short text string encoding its content properties. For music: track name and artist name. For books: title and author name. We convert this text into a 384-dimensional vector using the pre-trained sentence encoder `all-MiniLM-L6-v2` [8, 7]. Items that are semantically similar produce similar vectors; items that are stylistically distant produce different vectors. Critically, this embedding is derived from item metadata text, not from interaction patterns. A niche item can be embedded as readily as a popular one; representation quality depends on the informativeness of the metadata rather than on interaction frequency. All embeddings are computed once offline and cached.

Formally, $\phi(i)$ is the embedding of item i. For user u with history $H_u = \{i_1, \dots, i_n\}$ we compute the embedding matrix $E_u \in \mathbb{R}^{n \times d}$, where each row is the embedding of one item.

3.4 Interest Profile Clustering

We apply K-means clustering to E_u with a per-dataset K (selection procedure below), yielding centroids $\{c_1, \dots, c_K\}$. Each centroid represents a distinct interest profile: the semantic centre of gravity of one coherent facet of the user's taste. A user who consumes jazz, indie rock, and ambient electronica might, for example, end up with one centroid per genre under $K = 3$. The K centroids together form the user's interest profile $P_u = \{c_1, \dots, c_K\}$.

This is the core differentiator from average-embedding approaches. The mean of all embeddings collapses all interests into a single blurred point that sits between clusters and represents none of them well. K-means preserves the separation between interest modes, enabling targeted retrieval against each mode independently.

K is selected per dataset on a validation split carved from the training data (the last 20% of each user's chronological training history; the held-out test set is never touched during selection). K is swept over $\{2, 3, 4, 5, 8, 10, 15, 20, 30, 50\}$ (log-spaced above $K = 8$), and the sweep for a given dataset stops at the first K where either (i) validation Recall@10 improves by less than 2% relative to the previous tested K, or (ii) the mean number of items per cluster (training items per user, divided by K, averaged over users with enough history to cluster) falls below 3 — whichever triggers first. The selected K is the validation-Recall@10-maximizing point among the values tested before the stopping rule fires. This yields $K = 10$ for Last.fm-1K and Amazon Books, and $K = 15$ for MovieLens-25M. One methodological note on how the sweep was conducted: the stopping rule was applied moving forward from $K = 8$ (an earlier, already-reported checkpoint in this project's revision history) rather than from $K = 2$; applied retroactively to the $K \in \{2, \dots, 5\}$ region, the rule would in fact have triggered earlier for Amazon Books and MovieLens-25M, since validation Recall@10 dips slightly between some of those early, closely-spaced values before climbing again at higher K. We report this plainly as a scope decision — continuing the already-established $K = 8$ checkpoint forward, rather than restarting the rule from $K = 2$ — not a result the stopping rule itself produced.

For users with fewer than K items, Essence falls back to the content baseline (single mean embedding) rather than clustering with a reduced K, since scikit-learn's `KMeans` requires $n \geq K$ samples. In practice this fallback was not triggered for any user at any validated K used in this study: the minimum per-user training-history size is 40 (Last.fm-1K), 16 (Amazon Books), and 16 (MovieLens-25M), and every user on every dataset exceeds that dataset's validated K (10, 10, and 15 respectively). `KMeans` is run with scikit-learn's default `k-means++` initialization and `max_iter=300`, with `n_init=10` and `random_state=42` fixed across all runs; empty-cluster reassignment, when it occurs, is handled internally by scikit-learn's implementation.

3.5 Active Cluster Selection

At inference time, Essence selects the active centroid c_k^* as the cluster centre closest to the mean embedding of the user's r most recent training-set interactions, ordered chronologically (default $r = 10$):

$$c_k^* \ = \ \arg\min_k \left\| c_k - \mu(\phi(h_{n-r+1}), \ldots, \phi(h_n)) \right\|_2 \tag{1}$$

This mechanism assumes recent activity is informative about which interest cluster is currently active. We test this assumption directly via ablation (Section 5.8): replacing the recency-based query with the mean embedding of the user's entire training history degrades long-tail recall substantially on Last.fm-1K, indicating that recency carries signal beyond what the K-means clustering step alone provides. The ablation is the basis for retaining recency as the default mechanism.

3.6 Recommendation Generation

Given the active centroid c_k^*, Essence retrieves the top-M unseen items by cosine similarity:

$$R_u \ = \ \text{top-}M \ \{ \, i \in I \setminus H_u \ : \ \cos(\phi(i), \, c_k^*) \} \tag{2}$$

This retrieval step operates entirely in embedding space with no collaborative signal. An item's probability of appearing in R_u is determined solely by its latent similarity to the user's current interest cluster, not by how many other users have interacted with it.

3.7 Architecture Overview

Figure 1 illustrates the full Essence pipeline from raw user history to final recommendations.

4 Experimental Setup

4.1 Datasets

Last.fm-1K (music). Contains listening histories from 992 users across approximately 19 million events [9]. After filtering to users with 50–500 unique track interactions and removing duplicate (user, track) pairs, our evaluation subset contains 99 users and 22,767 unique tracks — this is the full filtered dataset, not a subsample: no further sampling is applied after the interaction-count filter. A positive interaction is any recorded play; there is no minimum play count or other threshold beyond the 50–500 per-user interaction-count filter. Item embeddings are built from track title + artist name (metadata only; no interaction signal enters the embedding).

Amazon Books (e-commerce). The Amazon Reviews 2023 Books 5-core dataset [10], which guarantees every user and item has at least 5 global interactions. After filtering to users with 20–200 unique item interactions and subsampling to 2,000 users (seed 42), our evaluation subset contains 2,000 users and 61,727 unique items. Item embeddings use title + author name + description, following the same metadata-only approach as Last.fm-1K. A positive interaction

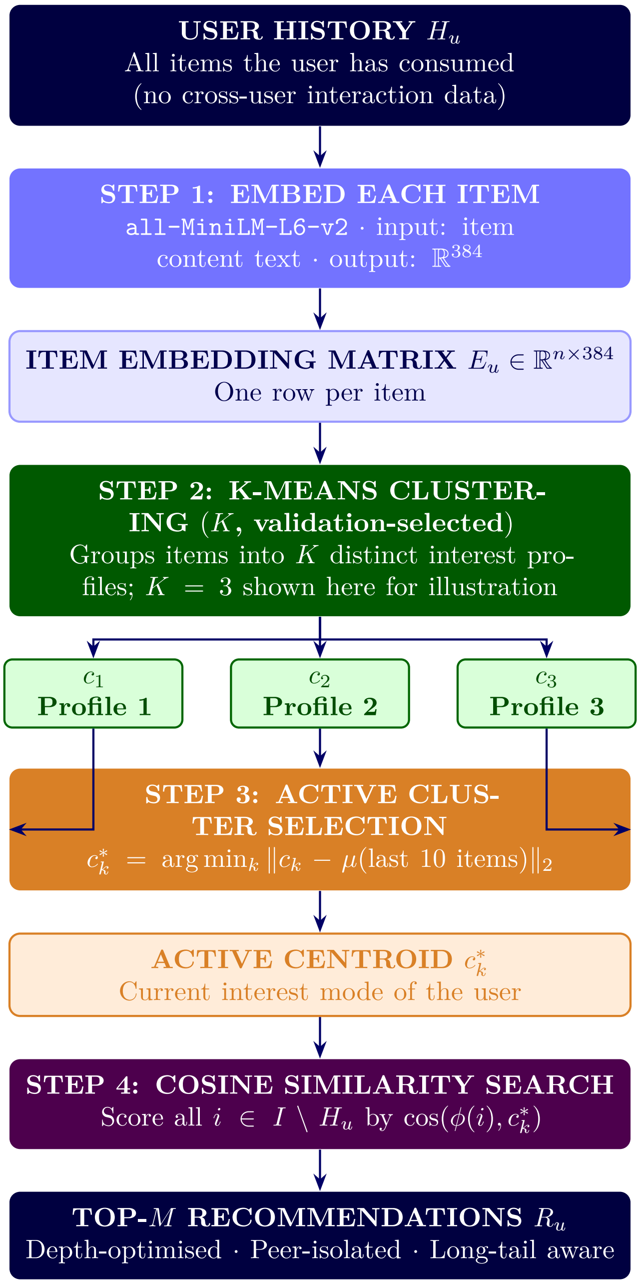

Figure 1: Essence recommendation pipeline. No cross-user interaction data is consulted at any stage (peer-isolation property).

is any rating or review in the 5-core file; no minimum star-rating value is required (a 1-star and a 5-star rating both count equally as an interaction). The 20–200-interaction filter and the 2,000-user subsample size are applied identically, with the same seed, to MovieLens-25M below — fixed a priori and held constant across both datasets rather than tuned per dataset, to keep the evaluation (including MIND/ComiRec training and the full bootstrap/FDR suite) computationally tractable at the same scale on both.

MovieLens-25M (film). A 2,000-user subsample (seed 42, same 20–200-interaction filter and sampling convention as Amazon Books) drawn from the MovieLens-25M ratings dataset. Item text is built from title + genres, plus overview text from The Movie Database (TMDb) when available; of 7,724 candidate items identified from the sampled users' interactions, 7,654 had a TMDb record successfully fetched at all (99.1%), and these form the final catalog — the remaining 70 items, for which the TMDb API lookup itself failed, are excluded from the candidate pool entirely (not merely missing overview text). Within the 7,654 fetched items, any with an empty overview field fall back to title + genres only rather than being dropped. Our evaluation subset contains 2,000 users and 7,654 catalog items. A positive interaction is any rating in the sampled ratings file; no minimum star-rating value is required.

Data attribution. This product uses the TMDb API but is not endorsed or certified by TMDb.

License and terms of use, verified against each source's own published terms. *Last.fm-1K* [9] — distributed with permission of Last.fm for non-commercial use only, per the dataset's official page; commercial use requires contacting Last.fm directly. *MovieLens-25M* — GroupLens's usage license prohibits redistribution without separate permission and prohibits commercial or revenue-bearing use without permission; research use is permitted with citation and no claim of University of Minnesota endorsement. *Amazon Reviews 2023 (Books 5-core)* [10] — no explicit license file or tag is published on the dataset's official site or Hugging Face card as of this writing; the only stated usage term is citation of the originating paper, so redistribution/commercial terms cannot be independently verified and none are claimed here. `all-MiniLM-L6-v2` — Apache 2.0 (permissive, commercial use allowed), per its Hugging Face model card. The accompanying repository includes catalog-level metadata (item titles, genres, authors, descriptions, and, for MovieLens-25M, TMDb overview text) and our own code, but not row-level user interaction data for MovieLens-25M or Amazon Books: those train/test files are regenerated locally via the included preprocessing scripts from each dataset's original source download rather than redistributed directly, given GroupLens's explicit redistribution restriction quoted above and Amazon's unverifiable license. An earlier revision of this repository had tracked those files directly; this was corrected, though the pre-correction commits remain retrievable from the repository's git history. Last.fm-1K's row-level interactions remain included, consistent with its license, which has no equivalent redistribution prohibition.

4.2 Train/Test Split

For each user on all three datasets, interactions are sorted chronologically and split per-user: the first 80% form the training set, and the final 20% form the test set. This chronological split is required for internal consistency with the active-cluster-selection mechanism (Section 3.5), which defines "recent" interactions in terms of training-set recency; a random split would make this definition incoherent, since a randomly held-out test item could chronologically precede items retained in training. The split is computed independently per user, with no shared global time cutoff across users: each user's train/test boundary is defined purely by their own interaction history, not by any dataset-wide date. This preserves within-user chronology but does not enforce a global temporal cutoff; possible cross-user temporal leakage for population-based baselines (CF, Popularity) — an item's co-occurrence signal can include other users' interactions that occurred, in wall-clock time, after a given user's personal train/test cutoff — is therefore a limitation of this evaluation protocol, not eliminated by it (Section 7).

Item embeddings are computed once, offline, over the full item catalog (train ∪ test), so that no test item is excluded from the retrieval candidate pool due to missing embedding coverage. This is independent of the train/test split decision and applies identically under either splitting strategy.

4.3 Long-Tail Definition

Long-tail items are those with insufficient interaction signal for collaborative filtering to represent reliably. We define long-tail operationally per dataset, anchored to each dataset's interaction density:

- **Last.fm-1K.** Items with a training-set interaction count of exactly 1 (*singleton tracks*): 16,761 of the 18,679 items appearing in the training set (89.7%). The full candidate pool (train ∪ test) is 22,767 tracks.

- **Amazon Books.** Items with a training-set interaction count of exactly 1 (*singleton items*): 42,878 of the 50,924 items appearing in the training set (84.2%). The full candidate pool (train ∪ test) is 61,727 items. Despite the 5-core guarantee of at least 5 global interactions, subsampling to 2,000 users reduces most items to a single observed interaction in the subset's training split.

- **MovieLens-25M.** Items with a training-set interaction count of exactly 1: 2,337 of the 6,745 items appearing in the training set (34.6%). Unlike Last.fm-1K and Amazon Books, where singleton items concentrate at the rare-popularity end of the catalog (deciles 3–9), MovieLens-25M's singletons concentrate more centrally, in deciles 2–5 (32.7% and 32.8% of all singletons falling in deciles 3 and 4 alone) — a dataset-specific difference in decile geometry, not a methodological inconsistency.

LT-Recall@10 is computed only over users with at least one long-tail test item; the remaining users contribute no LT-Recall@10 observation for that metric. Eligible users: 92 of 99 on Last.fm-1K, 1,292 of 2,000 on Amazon Books, 490 of 2,000 on MovieLens-25M.

Generalisation principle. The long-tail threshold should be set at the point where collaborative filtering loses reliable signal, empirically where item interaction count falls below the minimum required for stable co-occurrence estimation. On Last.fm-1K and Amazon Books, the singleton threshold (train interaction count = 1) proved the most meaningful boundary: alternative thresholds either collapsed toward the full catalog (losing discriminative power) or excluded too few items; the same threshold was applied to MovieLens-25M without modification and produced a comparably non-degenerate singleton population (34.6% of training-set items), though alternative thresholds were not separately swept on this third dataset. The key invariant is that long-tail items are defined on the *training set only*, after the train/test split, ensuring no leakage from held-out interactions into the long-tail classification.

LT-Recall@10 is computed only over test items falling in the long-tail set. Users with no long-tail test items are excluded from this metric.

4.4 Systems Compared

- **Random.** Recommends M items drawn uniformly from the unseen candidate pool. Establishes the floor against which all other systems are compared.

- **Popularity.** Recommends the M globally most-interacted-with items the user has not consumed. A non-personalized baseline; included for comparison but not treated as collaborative filtering.

- **CF (ItemKNN).** An item-based collaborative filtering baseline, computed via cosine similarity over the sparse user-item interaction matrix. Recommends items most similar, by co-occurrence pattern, to items the user has already consumed. No neighbor-count cutoff or shrinkage term is applied: a candidate's score is the cosine-weighted sum over all training items with any co-occurring user, i.e. full-neighborhood ($k = \infty$) ItemKNN rather than a truncated top-k variant.

- **Content (Average Embedding).** Computes the mean of all the user's item embeddings and retrieves the M most similar unseen items by cosine similarity. Standard content-based filtering without interest decomposition.

- **Essence (validation-selected K).** Clusters user history into K centroids ($K = 10$ Last.fm-1K/Amazon Books, $K = 15$ MovieLens-25M; selection procedure in Section 3), selects the active centroid by Equation (1), and retrieves top-M unseen items by Equation (2).

- **Last-Item.** Recommends the M unseen items most similar, by cosine similarity, to the single most recent item in the user's training history. The simplest possible recency-based retrieval: no averaging, no decay, no clustering.

- **Avg-Last-10.** Recommends the M unseen items most similar to the uniform mean embedding of the user's last 10 training interactions.

- **Recency-Weighted.** Recommends the M unseen items most similar to an exponentially-decay-weighted mean of the user's last 10 training interactions (decay = 0.9; the most recent item receives weight $\propto 0.9^0$, the tenth-most-recent $\propto 0.9^9$, renormalized to sum to 1). Last-Item, Avg-Last-10, and Recency-Weighted use exactly the same item embeddings as Content and Essence; the only difference across all five is how the query vector is constructed from recent history.

- **MIND.** A hand-implemented dynamic-routing multi-interest network, following Li et al.'s original architecture [1] ($K = 4$ interest capsules, a bilinear routing matrix shared across capsules, 3 routing iterations, routing logits initialized $b \sim \mathcal{N}(0, 0.01^2)$). Neither MIND nor ComiRec is available in standard recommendation libraries, and neither original paper's reported benchmark numbers are directly comparable here: Li et al. benchmark on an internal industrial dataset and Amazon Books, and Cen et al. on Amazon Books, Taobao, and a mobile-Taobao set, all under sampled-negative evaluation (one positive against roughly 100–1,000 negatives); none report Last.fm-1K at all, and Amazon Books here uses this paper's own 2,000-user subsample, full-catalog ranking, and no negative sampling (Section 4) — a harder protocol under which lower absolute recall than the original papers' is expected by construction, not evidence of a broken implementation. We therefore verify correctness against each model's architectural specification directly, rather than by matching a published benchmark number: a synthetic conformance test for MIND's routing mechanism, and a directional sanity check (Recall@10 convincingly above Random and Popularity, and within a plausible order of magnitude of the published ranges once the protocol difference is accounted for) applied to both models' final results. The conformance test caught a symmetry-breaking bug in *our implementation* — not a flaw in the original architecture: an initial version zero-initialized b, which, because the shared bilinear matrix makes the pre-routing representation identical across capsules, guaranteed every capsule updated identically, silently collapsing the model's effective interest count to 1 regardless of the nominal K. The test (`test_mind_routing.py`) caught this before any number reported here was generated; catching a real, non-obvious bug is the evidence the verification method works, not just a disclosed limitation.

- **ComiRec (SA).** A hand-implemented self-attentive multi-interest network, following Cen et al.'s original architecture [2] ($K = 4$ interest capsules, per-capsule learned query projections implemented as a single K-output linear layer, softmax attention over sequence positions). ComiRec's per-capsule projections are independently initialized rather than derived from a shared matrix, so it has no equivalent to MIND's symmetry-breaking failure mode and was checked with the directional sanity check described above rather than a dedicated conformance test; no comparable bug was found.

- **MIND/ComiRec training.** Both models share the same fixed, untuned training procedure, taken from the original papers' recommended ranges rather than grid-searched: Adam optimizer, learning rate 10^{-3}, batch size 64, max sequence length 50, up to 50 epochs with early stopping (patience 5) on validation loss over each user's held-out last training interaction. The training objective is a sampled-softmax loss: the positive item's score plus 100 uniform-random negative items' scores (drawn item-uniformly, excluding the padding index) are passed through a cross-entropy loss against the positive. Item and interest-capsule embeddings are 384-dimensional, matching the shared `all-MiniLM-L6-v2` embedding space used by every other system.

Both MIND and ComiRec are trained across 3–4 seeds per dataset to confirm the domain-dependence pattern (Section 5.2) is not seed-specific (Section 5.3). Paired statistical inference for MIND and ComiRec is conditional on each model's canonical (seed-42) training run; the multi-seed analysis (Section 5.3) evaluates whether the qualitative ordering is robust to initialization, not training-seed variance within the bootstrap itself.

4.5 Metrics

What we are measuring. Each system produces a ranked list of 10 recommended items per user. We know which items that user actually consumed in the test set (ground truth). The evaluation asks: how much of that ground truth did the system recover?

This is a retrieval problem, not a classification problem. All systems are evaluated against the *full item catalog* with no negative sampling, following the full-ranking evaluation protocol [11]. This also avoids a separate, distinct pitfall: Rendle [19] shows that sampled evaluation metrics are not merely noisier versions of their exact counterparts but can invert which of two systems appears better, a risk this full-catalog protocol is structurally immune to regardless of sample size chosen. Absolute recall values are low by construction under this protocol: the meaningful signal is relative comparison across systems evaluated under identical conditions.

- **Recall@10.** Fraction of the user's test items appearing in the top-10 recommendations.

$$\text{Recall@10} = \frac{|\,R_u \cap \text{Test}_u\,|}{|\,\text{Test}_u\,|}$$

  Example: 20 test items, 2 hits in top-10. Recall@10 = 0.10. Averaged across all users.

- **Long-Tail Recall@10 (LT-Recall@10).** Recall@10 restricted to singleton test items only.

$$\text{LT-Recall@10} = \frac{|\,R_u \cap \text{LT-Test}_u\,|}{|\,\text{LT-Test}_u\,|}$$

  A long-tail hit means the system recommended a niche item that this specific user genuinely consumed, without any popularity signal and without any other user's behaviour to guide it. This metric directly tests Essence's core hypothesis.

- **Lift over random.** Recall@10 divided by expected random recall ($M/|I|$). Normalises for candidate pool size and makes cross-dataset comparison interpretable.

Both Recall@10 and LT-Recall@10 are macro-averaged: the per-user ratio is computed first, then averaged across users (LT-Recall@10 over eligible users only — those with at least one long-tail test item — rather than pooling hits and opportunities across users before dividing). Ties in the top-M ranking are broken deterministically, though not by an identical mechanism across datasets: by ascending item index for Amazon Books/MovieLens-25M's vectorized scoring, and by embedding-map iteration order for Last.fm-1K's dictionary-based scoring. The Random baseline draws a single deterministic sample per user from a per-user seed (MD5 hash of the user ID), not an average over repeated draws.

4.6 Implementation

All systems implemented in Python 3.11. Embeddings via `sentence-transformers` (`all-MiniLM-L6-v2`, 384 dims). K-means via `scikit-learn` (`n_init=10`, `random_state=42`). MIND and ComiRec are implemented from scratch in PyTorch, since neither is available in RecBole or comparable standard recommendation libraries; both are trained per dataset. All ranking steps break score ties deterministically (stable sort by item index), so reported results are exactly reproducible; this matters for sparse baselines such as CF (ItemKNN), where many candidate items receive identical scores. Significance testing methodology (paired bootstrap, FDR correction, effect sizes, BCa cross-checks) is described once, in Section 5.6, and applied identically across all three datasets rather than being re-derived per dataset. Code and data available at: `https://github.com/biditdas18/essence`

5 Results

5.1 Headline Results, All Three Datasets

Table 3 and Table 4 report Recall@10 and LT-Recall@10 for all ten systems on all three datasets (99 users on Last.fm-1K, 2,000 users on Amazon Books, 2,000 users on MovieLens-25M; chronological 80/20 split per user throughout; catalog sizes and long-tail definitions in Section 4). Essence's rank among the ten systems is noted next to its own value in each column.

Table 3: Headline Recall@10, all three datasets ($M = 10$). Best result per dataset in **bold**. Essence's rank (of 10) is shown in parentheses. Essence uses its validation-selected K per dataset ($K = 10$ Last.fm-1K/Amazon Books, $K = 15$ MovieLens-25M; Section 3).

System	**Last.fm-1K**	**Amazon Books**	**MovieLens-25M**
Random	0.0001	0.0001	0.0012
Popularity	0.0023	0.0021	0.0517
CF (ItemKNN)	0.0018	0.0031	**0.0850**
Content (Avg Embedding)	0.0038	0.0173	0.0043
Last-Item	0.0265	**0.0311**	0.0091
Avg-Last-10	0.0252	0.0235	0.0064
Recency-Weighted	**0.0304**	0.0260	0.0076
MIND	0.0078	0.0052	0.0423
ComiRec (SA)	0.0137	0.0056	0.0706
Essence (val. K)	0.0281 (2nd)	0.0253 (3rd)	0.0053 (8th)

Table 4: Headline LT-Recall@10, all three datasets ($M = 10$). Best result per dataset in **bold**. Essence's rank (of 10) is shown in parentheses. Essence uses its validation-selected K per dataset, as above.

System	Last.fm-1K	Amazon Books	MovieLens-25M
Random	0.0000	0.0000	0.0000
Popularity	0.0000	0.0000	0.0000
CF (ItemKNN)	0.0000	0.0137	0.0037
Content (Avg Embedding)	0.0029	0.0124	0.0020
Last-Item	0.0064	**0.0215**	0.0031
Avg-Last-10	0.0134	0.0196	0.0026
Recency-Weighted	**0.0151**	0.0179	**0.0046**
MIND	0.0021	0.0057	0.0000
ComiRec (SA)	0.0050	0.0121	0.0014
Essence (val. K)	0.0142 (2nd)	0.0207 (2nd)	0.0030 (4th)

Essence's standing is not uniform across datasets. On Amazon Books it is 3rd of 10 on Recall@10 and 2nd of 10 on LT-Recall@10 (behind only Last-Item on both metrics; not significantly different from Recency-Weighted on Recall@10 — Section 5.6); on Last.fm-1K it is 2nd of 10 on both Recall@10 and LT-Recall@10, not significantly different from any of the three recency-only baselines (numerically ahead of Last-Item and Avg-Last-10, behind Recency-Weighted); on MovieLens-25M it is 8th of 10 on Recall@10, ahead of only Content and Random, and 4th on LT-Recall@10. Full significance testing for every pairwise comparison in these tables is reported in Section 5.6.

5.2 The Domain-Dependence Pattern

Essence's rank is not randomly scattered across the three datasets: it is associated with the strength of collaborative and popularity signal available in each domain, measured by how well the non-personalized Popularity baseline and the CF (ItemKNN) baseline perform in that domain (Figure 2, Table 5). Establishing this association as predictive would require additional datasets and an independently-defined domain property, not one measured post hoc via baseline performance, as here.

Where the evaluated collaborative baselines perform weakly, Essence sits in the upper half of the field on both datasets. On Amazon Books it significantly beats CF-ItemKNN, the non-personalized baselines, Content, and both neural multi-interest baselines on Recall@10 (Cohen's d 0.099–0.282); it is not significantly different from Recency-Weighted and significantly behind only Last-Item. On Last.fm-1K, at its validation-selected $K = 10$, Essence significantly beats CF-ItemKNN, the non-personalized baselines, Content, MIND, *and* ComiRec on Recall@10 (Cohen's d 0.220–0.301) — at $K = 3$, Essence's Recall@10 was not significantly different from ComiRec's (0.0119 vs. 0.0137); at the validation-selected K, Essence's own Recall@10 rises to 0.0281 and the non-significant difference resolves into a significant win. Essence is not significantly different from any of the three recency-only baselines on Last.fm-1K, including Recency-Weighted, despite trailing it numerically. (LT-Recall@10 shows no significant differences in either direction on Last.fm-1K at this sample size, $n = 99$; see Section 5.10.) Where the evaluated collaborative baselines perform strongly (MovieLens-25M, driven almost entirely by densely-rated popular titles: item-based CF reaches 0.0850 Recall@10, the single highest value recorded in this study), Essence falls to 8th of 10, losing to CF, Popularity, and both neural baselines by the four largest-magnitude effect sizes observed anywhere in this evaluation (Cohen's d from -0.378 to -0.550, all four significant after BH-FDR correction; every other significant

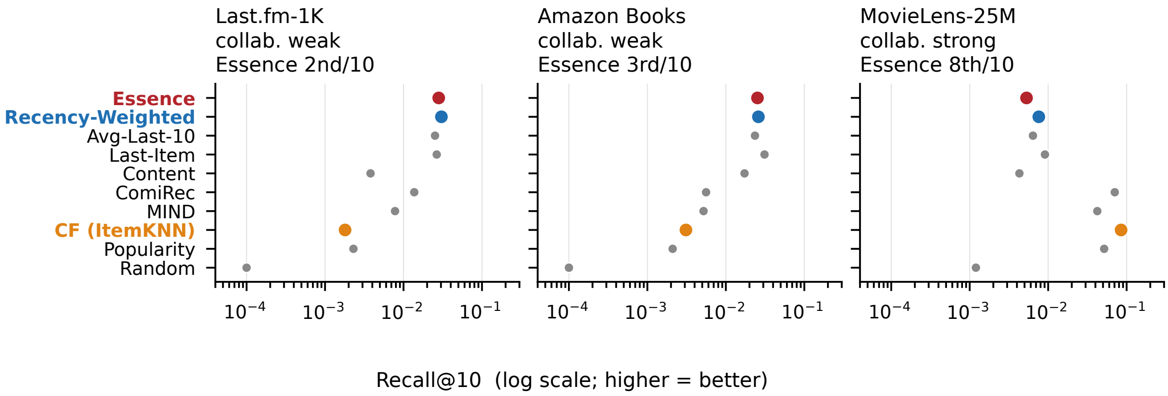

Figure 2: Recall@10, all ten systems, all three datasets (log scale). System order is identical in every panel, so the reordering *between* panels is the domain-dependence finding: Essence (red) sits near the top where the collaborative baselines are weak and below them where CF-ItemKNN (orange) is strongest. Recency-Weighted (blue), which performs no clustering, tracks Essence throughout — the visual form of the clustering-null result. Point estimates only; significance and effect sizes in Tables 3–4.

Table 5: Essence's rank (of 10 systems, Recall@10) against the performance of the evaluated collaborative baselines in each domain. "n.s." = not significantly different (paired bootstrap). Ranks are ordinal positions on the Recall@10 point estimate; the significance statements are paired-bootstrap results after BH-FDR correction and do not follow from rank order. A system Essence ranks above is not necessarily one it significantly beats, which is why the two are reported separately here rather than as a single "beats n" count.

Dataset	**Essence rank**	**CF / Popularity performance**
Amazon Books	3rd of 10 by rank. Significantly ahead of CF-ItemKNN, the non-personalized baselines, Content, and both neural multi-interest baselines; n.s. vs. Recency-Weighted; significantly behind Last-Item	Weak (CF 0.0031, Popularity 0.0021)
Last.fm-1K	2nd of 10 by rank. Significantly ahead of CF-ItemKNN, the non-personalized baselines, Content, MIND and ComiRec; n.s. vs. all three recency-only baselines, including Recency-Weighted	Weak (CF 0.0018, Popularity 0.0023)
MovieLens-25M	8th of 10 by rank. Ranks above Content and Random only	**Strong** (CF/Pop./ComiRec/MIND are top 4)

effect size across Last.fm-1K and Amazon Books stays at or under $|d| \approx 0.30$).

This pattern is consistent and well-evidenced across all three datasets. We treat it as a genuine domain-dependence finding rather than an architectural strength: it is Essence's *relative standing* that is associated with domain conditions, not necessarily anything specific to the K-means clustering mechanism as opposed to the underlying content embeddings more generally (Section 5.10 addresses this directly).

Is domain-dependence specific to Essence's clustering? A direct check: Recency-Weighted uses the identical item embeddings as Essence but no K-means clustering at all, so comparing its own win/loss pattern against CF-ItemKNN, Popularity, MIND, and ComiRec isolates whether the domain-dependence property above is something clustering adds, or something already present in content-embedding/recency-based retrieval regardless of clustering. On Recall@10, the answer is unambiguous: Recency-Weighted shows the identical significance pattern Essence does against all four baselines, on all three datasets (paired bootstrap, 10,000 resamples, BH-FDR within the same family structure as Section 5.6: Last.fm-1K/Amazon Books pooled as one 16-comparison family, MovieLens-25M as its own 8-comparison family). Where collaborative signal is weak, both significantly beat all four baselines on both datasets (Last.fm-1K: Essence $d \in [0.220, 0.281]$, Recency-Weighted $d \in [0.250, 0.299]$; Amazon Books: $d \in [0.220, 0.252]$ vs. $d \in [0.216, 0.248]$; 4 of 4 comparisons significant for each system, each dataset). Where it is strong (MovieLens-25M), both lose significantly to all four (Essence $d \in [-0.550, -0.378]$, Recency-Weighted $d \in [-0.539, -0.349]$; 4 of 4 each). Recency-Weighted's own rank tracks the same direction as Essence's across the three datasets: 1st of 10 on Last.fm-1K, 2nd on Amazon Books, falling to 6th on MovieLens-25M. Domain-dependence on Recall@10 is therefore a property of the broader no-cross-user-pooling architecture family — content embeddings retrieved by recency, clustered or not — not something specific to K-means clustering.

On LT-Recall@10 the pattern does not replicate as cleanly. Recency-Weighted's long-tail win pattern against the same four baselines is flatter across datasets: 2 of 4 significant wins on Last.fm-1K ($d \in [0.109, 0.174]$), 2 of 4 on Amazon Books ($d \in [0.030, 0.157]$), and 2 of 4 even on MovieLens-25M ($d \in [0.011, 0.082]$) — compared to Essence's 0 of 4, 3 of 4, and 0 of 4 respectively. Recency-Weighted retains long-tail wins on MovieLens-25M where Essence has none, so its long-tail domain-dependence is weaker than Essence's, not identical to it. More directly relevant to whether clustering itself contributes anything: Essence and Recency-Weighted are not significantly different from each other on LT-Recall@10 on any of the three datasets ($d = -0.051$, $q = 0.643$, Last.fm-1K; $d = +0.024$, $q = 0.446$, Amazon Books; $d = -0.045$, $q = 0.376$, MovieLens-25M) — consistent with, and reinforcing, the clustering-null result already reported in Section 5.10: K-means clustering shows no detectable long-tail advantage over simple recency-weighting on any dataset tested, weak-collaborative-signal or strong.

The practical implication is direct. Since clustering is not currently contributing any measurable long-tail advantage over Recency-Weighted alone, on any of the three datasets, future work aimed at closing a long-tail recall gap is unlikely to succeed by refining the interest-selection or clustering mechanism specifically; the embedding representation itself, or the retrieval mechanism built on top of it, is the more promising target (Section 7).

5.3 MIND/ComiRec Multi-Seed Robustness

Table 6 reports MIND and ComiRec Recall@10 across independent training seeds per dataset (the canonical seed 42 plus 2–3 additional seeds), to check whether the domain-dependence pattern above depends on a single training run.

Table 6: MIND/ComiRec Recall@10 across training seeds. "Beats Essence" counts seeds where the baseline's Recall@10 exceeds Essence's own Recall@10 for that dataset (Table 3).

Dataset (Essence Recall@10)	System	Mean ± Std	n	Beats Essence
Last.fm-1K (0.0281)	MIND	0.0067 ± 0.0014	4	0/4
	ComiRec (SA)	0.0116 ± 0.0020	4	0/4
Amazon Books (0.0253)	MIND	0.0053 ± 0.0004	4	0/4
	ComiRec (SA)	0.0064 ± 0.0007	4	0/4
MovieLens-25M (0.0053)	MIND	0.0464 ± 0.0036	3	3/3
	ComiRec (SA)	0.0607 ± 0.0086	3	3/3

Essence's Recall@10 in this table uses its validation-selected K per dataset (Table 3), not the $K = 3$ value used in an earlier revision of this analysis; at $K = 3$, ComiRec (SA) beat Essence on 2 of 4 Last.fm-1K seeds, which this paper originally characterized as not significantly different. At the validation-selected $K = 10$, Essence's own Recall@10 rises enough ($0.0119 \rightarrow 0.0281$) that ComiRec no longer beats it on any seed. The domain-dependence pattern itself is not an artifact of a single seed: MIND never outranks Essence on Last.fm-1K or Amazon Books and always outranks Essence on MovieLens-25M, across every seed tested. ComiRec (SA) now shows the same clean pattern on all three datasets (never on Last.fm-1K or Amazon Books, always on MovieLens-25M) rather than being seed-dependent anywhere.

5.4 An Additional Collaborative Baseline: BPR-MF

The domain-dependence pattern above rests on comparisons against a single collaborative-filtering baseline, CF-ItemKNN, throughout. To check that the pattern is not an artifact of that particular implementation, we additionally implement BPR-MF [14] (Bayesian Personalized Ranking Matrix Factorization): a pairwise-ranking loss over learned latent user and item factors, optimized by stochastic gradient descent to rank each user's observed positive items above randomly sampled negatives. This is structurally unrelated to ItemKNN's neighborhood-based scoring — the two share no code and no architectural assumptions — while remaining, like ItemKNN, a genuine collaborative method: representations are learned entirely from cross-user interaction data, with no item content used at any stage.

Before trusting any real-dataset number from this new implementation, we ran a dedicated conformance test on synthetic data, the same discipline that caught the MIND routing bug in Section 4: (1) training loss must decrease with training rather than staying flat or increasing; (2) given synthetic data with a known latent two-block structure (two disjoint user/item clusters with no cross-block interactions), the model must recover that structure, ranking each block's own items higher for that block's users than the other block's items; (3) a user's single known positive item must be recovered near the top of an otherwise-random ranking, as a check against index-mapping bugs. The conformance test itself needed one correction before all three checks passed cleanly: an initial, arbitrarily strict pass/fail threshold on the loss-decrease check (a required percentage drop within a fixed number of epochs) flagged a false failure even though the model was training correctly, just more slowly than that threshold assumed; the check was rewritten to a more principled criterion (loss below the untrained baseline, and continuing to decrease with more training) rather than the threshold being loosened to make the existing test pass. This was a bug in the test, not in the model. Hyperparameters are fixed, not tuned — 32 latent factors, learning rate 0.05, L2 regularization 0.01, 30 epochs, seed 42 — matching how CF-ItemKNN is treated as an untuned baseline elsewhere in this paper.

Table 7: BPR-MF vs. Essence, all three datasets, using the same paired bootstrap protocol used throughout this paper (10,000 resamples, seed 42), reported as raw p-values, not corrected for multiple comparisons within a family as the paper's primary baseline comparisons are (Section 5.6). Recall@10 differences are significant ($p < 0.05$) on all three datasets; LT-Recall@10 differences are significant on Last.fm-1K and Amazon Books but not on MovieLens-25M ($p \approx 0.09$).

Dataset	**Essence R@10**	**BPR-MF R@10**	**Essence LT-R@10**	**BPR-MF LT-R@10**
Last.fm-1K	0.0281	0.0005	0.0142	0.0000
Amazon Books	0.0253	0.0015	0.0207	0.0000
MovieLens-25M	0.0053	0.0647	0.0030	0.0000

The result replicates the domain-dependence pattern exactly, with a collaborative method that shares nothing but the underlying data requirement with CF-ItemKNN. Where collaborative signal is weak, Essence beats BPR-MF (raw $p < 0.05$, uncorrected; Last.fm-1K: $d = +0.296$; Amazon Books: $d = +0.261$). Where collaborative signal is strong, Essence loses to it (raw $p < 0.05$, uncorrected; MovieLens-25M: $d = -0.518$, comparable in magnitude to the $d = -0.550$ loss to CF-ItemKNN); BPR-MF there ranks immediately behind CF-ItemKNN and ComiRec (SA) among all ten systems tested (Table 3). BPR-MF's own LT-Recall@10 is 0.0000 on all three datasets, including MovieLens-25M where it wins decisively on Recall@10 — another instance of a pattern discussed elsewhere in this Discussion, that strong aggregate collaborative performance does not translate into long-tail recovery. This does not establish that domain-dependence generalizes to collaborative methods in general — two implementations is not an exhaustive survey of the collaborative-filtering family — but it is evidence against the narrower concern that the finding is specific to one, possibly unrepresentative, CF baseline.

BPR-MF's significance tests are reported above as raw p-values rather than folded into one of this paper's existing FDR-corrected families (Section 5.6): BPR-MF was added as an ad hoc, late-stage robustness probe, not part of the pre-specified evaluation design those families were built from, so it is disclosed standalone rather than artificially forced into a family it was never planned alongside. BPR-MF is an untuned, exploratory robustness baseline. Its strong MovieLens-25M performance reproduces the direction of the ItemKNN result (Section 7 discusses ItemKNN's own separate tuning check), but its performance on the other two datasets may be understated by the fixed configuration used here.

5.5 Overnight Robustness-Check Summary

Table 8 summarizes the four additional robustness checks referenced in the Limitations section: rating-threshold sensitivity, a global-timestamp-cutoff split, multi-seed resampling, and validation-based ItemKNN hyperparameter tuning. Sample-size changes matter for reading these numbers: the global-cutoff and rating-threshold checks each exclude some users, most severely for MovieLens-25M under the global cutoff (98.6% stranded), so those rows are more a test of the split protocol itself than a like-for-like comparison to the canonical numbers.

Table 8: Overnight robustness-check summary. "Essence R@10" compares the canonical per-user-split, seed-42 Recall@10 to each check's result; "CF R@10" (tuning row only) compares CF-ItemKNN's own Recall@10, since that is what was tuned, not Essence. "Conclusion preserved?" states whether both (i) Essence's baseline rank order and (ii) the clustering-null result (no stratum shows Essence significantly ahead of Recency-Weighted, BH-FDR corrected within each check's own family, degenerate zero-variance cells excluded) hold under the perturbation, computed from each check's already-collected per-user silhouette/recall data.

Check	Sample size / users affected	Recall@10 outcome	Conclusion preserved?
Rating-threshold (≥ 4)	Amazon: 1,602/2,000 (20% dropped); MovieLens: 1,432/2,000 (28% dropped)	Amazon: 0.0253 → 0.0300; MovieLens: 0.0053 → 0.0060	Yes
Multi-seed resampling (2 extra seeds)	Amazon/MovieLens: 2,000/2,000 each seed (different users, same count)	Amazon: 0.0253 → 0.0256–0.0258; MovieLens: 0.0053 → 0.0056–0.0066	Yes
Global-timestamp cutoff	Last.fm-1K: 60/99 (39% stranded); Amazon: 1,124/2,000 (44%); MovieLens: 28/2,000 (98.6%)	Last.fm-1K: 0.0281 → 0.0032; Amazon: 0.0253 → 0.0164; MovieLens: 0.0053 → 0.0014	Last.fm-1K: **No** (rank reverses; null still holds); Amazon: Yes; MovieLens: N/A (underpowered)
ItemKNN validation-based tuning	All users, all three datasets (no exclusions; CF-ItemKNN retuned)	CF R@10: Last.fm 0.0018 → 0.0005 (−69%); Amazon 0.0031 → 0.0062 (+98%); MovieLens 0.0850 → 0.0850 (0%)	Yes

5.6 Statistical Validation Methodology

All primary baseline and stratification comparisons reported in this paper use paired user-level bootstrap inference on the per-user difference between two systems (10,000 resamples over users, seed 42; observed mean difference, 95% percentile CI, and a paired Cohen's d computed as mean(difference)/std(difference)). Independent per-system confidence intervals are not used: they do not test the quantity of interest directly (whether one system's per-user metric exceeds another's) and can disagree with the paired test on borderline cases. Where multiple related comparisons are tested together, we apply Benjamini–Hochberg (BH) false-discovery-rate correction at $q < 0.05$ within that family, and cross-check every raw-significant result under bias-corrected and accelerated (BCa) bootstrap resampling to confirm the conclusion is not an artifact of the simpler percentile method.

Correction families. The Last.fm-1K/Amazon Books baseline comparison (Essence vs. each of the nine other systems, per dataset, per metric; 36 comparisons) and the multimodality-stratification test (Essence vs. Recency-Weighted, stratified by each user's own per-history silhouette score into low/medium/high clustering-quality tertiles, per dataset, per metric; up to 18 comparisons, Section 5.10) are corrected together as a single Benjamini–Hochberg family (54 comparisons nominally; 52–53 in practice, see below): pooling them changes no comparison's significance verdict relative to correcting them separately, so we report the simpler pooled version. The MovieLens-25M baseline comparison (18 comparisons, testing the same underlying question as the family above but not included in the pooling-invariance check performed on it) and the decile-level cold-start analysis (Section 5.7; Essence, and separately each recency-only baseline, vs. CF-ItemKNN, restricted to the rarest-item deciles) are each corrected as their own family: the cold-start analysis asks a narrower, mechanistically distinct question, isolating a single structural property of CF (zero co-occurrence signal for zero-training-interaction items) rather than aggregate performance. One correction to the stratification family's comparison count: one or two of the up-to-18 stratum/metric cells are occasionally degenerate — every user in that cell scores exactly zero on *both* Essence and Recency-Weighted (most often the low- or medium-silhouette stratum on the smaller LT-Recall@10 subsets), giving zero variance in the paired difference. The standard two-sided bootstrap p-value formula is undefined in this case and, applied naively, returns $p = 0$ (spuriously "significant"); these cells are excluded from the pooled family rather than included as evidence of a real difference, since there is no difference to detect. This yields 53 comparisons at $K = 3$ and 52 at the validation-selected K (Section 3), not 54 in either case.

5.7 Cold-Start vs. Singleton Long-Tail: A Scope Boundary

LT-Recall@10, as defined in Section 4, treats every training-set singleton as long-tail without distinguishing *how* singleton an item is. A finer-grained popularity-decile breakdown reveals that this hides two very different regimes.

Table 9: Essence vs. CF-ItemKNN at the two rarest popularity deciles, all three datasets, at Essence's validation-selected K per dataset. Positive diff favors Essence. Paired bootstrap, 10,000 resamples; BH-FDR within this 6-comparison family.

Dataset	**Decile**	**Essence**	**CF**	**Diff**	**Cohen's d**	**Sig. (FDR)**
Last.fm-1K ($K = 10$)	1	0.0221	0.0000	+0.0221	0.246	Yes
Last.fm-1K ($K = 10$)	2	0.0352	0.0000	+0.0352	0.351	Yes
Amazon Books ($K = 10$)	1	0.0215	0.0000	+0.0215	0.228	Yes
Amazon Books ($K = 10$)	2	0.0224	0.0004	+0.0220	0.193	Yes
MovieLens-25M ($K = 15$)	1	0.0092	0.0000	+0.0092	0.108	Yes
MovieLens-25M ($K = 15$)	2	0.0029	0.0047	−0.0019	−0.026	No

At true cold-start (decile 1: items with *zero* training-set interactions, not merely rare ones), Essence significantly beats CF-ItemKNN on all three datasets, and this pattern is unchanged from an earlier fixed-$K = 3$ version of this analysis (in which MovieLens-25M's decile-1 result was a marginal pass rather than a clean one; at the validation-selected $K = 15$ it is unambiguous, $q = 0.002$). But this is not evidence for Essence's architecture specifically: re-running the identical test with Recency-Weighted, Last-Item, or Avg-Last-10 in Essence's place, 15 of 18 comparisons are also significant, including every Last.fm-1K and Amazon Books comparison for all three non-Essence baselines (this check does not depend on Essence's K and was not re-run). The mechanism is mechanical, not architectural: CF-ItemKNN's co-occurrence score is *exactly* 0.0000 for any item with zero training interactions by construction, so any method that scores items via content embeddings — clustering or not — clears that bar trivially. The defensible claim is therefore narrower than "Essence solves cold-start": Essence, like every content-embedding-based method tested, beats collaborative filtering at true cold-start.

Separately, on Amazon Books, an earlier fixed-$K = 3$ version of this analysis found Essence losing to Last-Item at both deciles 1 and 2 (significant after FDR, $d = -0.093$ and -0.065) and to Recency-Weighted at decile 2 ($d = -0.075$). At the validation-selected $K = 10$, none of these comparisons remain significant: Essence vs. Last-Item is $d = -0.064$ (decile 1, $q = 0.082$) and $d = -0.022$ (decile 2, $q = 0.549$); vs. Recency-Weighted, $d = -0.003$ (decile 1, $q = 0.927$) and $d = -0.023$ (decile 2, $q = 0.549$); vs. Avg-Last-10, neither decile is significant at either K. Essence's cold-start weakness relative to the recency baselines specifically is therefore weaker evidence at the validation-selected K than it was at $K = 3$, though the direction (a numerical, non-significant disadvantage) is unchanged. The distributional claim that Essence's actual LT-Recall@10 hits on Amazon Books concentrate in deciles 3–9 has not been re-verified at $K = 10$ and is not repeated here pending that check. The decile-1/2 cold-start weakness and the singleton-recall headline win remain about largely disjoint item populations at the new K (Table 9 vs. Table 4): not a contradiction, a scope boundary. The long-tail recovery result in Table 4 should be read as covering moderately-rare-but-observed items, not true cold-start items.

5.8 Active Cluster Selection Ablation

Section 3.5 motivates recency-based active cluster selection on the grounds that it captures session-aware context without requiring a trained sequential model. We test this directly by comparing it against an alternative: selecting the active centroid using the mean embedding of the user's entire training history, rather than the most recent $r = 10$ interactions.

Table 10: Active cluster selection ablation, all three datasets at their validation-selected K (chronological split, $M = 10$).

Variant	Recall@10	LT-Recall@10	LT / Content
Last.fm-1K ($K = 10$)			
Essence (recency, $r = 10$)	0.0281	0.0142	4.84×
Essence (mean-select, full history)	0.0016	0.0005	0.16×
Content (Avg Embedding, reference)	0.0038	0.0029	1.0×
Amazon Books ($K = 10$)			
Essence (recency, $r = 10$)	0.0253	0.0207	1.67×
Essence (mean-select, full history)	0.0195	0.0118	0.95×
Content (Avg Embedding, reference)	0.0173	0.0124	1.0×
MovieLens-25M ($K = 15$)			
Essence (recency, $r = 10$)	0.0053	0.0030	1.46×
Essence (mean-select, full history)	0.0044	0.0037	1.83×
Content (Avg Embedding, reference)	0.0043	0.0020	1.0×

On Last.fm-1K, mean-select performs below even the Content baseline on long-tail recall (LT/Content 4.84× in recency-select's favor). This is consistent with the mechanism collapsing toward Content: selecting the centroid nearest a user's global mean embedding is functionally similar to querying with that mean directly, which is what Content already does, while incurring the additional cost of clustering without a corresponding benefit. On Amazon Books, the same pattern holds but is less extreme: recency-select still leads on every column, but mean-select stays above the Content baseline (0.95×–1.67×) rather than collapsing below it. On MovieLens-25M, the picture is mixed rather than one-sided: recency-select leads on raw Recall@10 (0.0053 vs. 0.0044), but mean-select is *ahead* on LT-Recall@10 (0.0037 vs. 0.0030, 1.83× vs. 1.46× Content). No significance testing has been run on these differences; they are reported as raw comparisons, consistent with how this ablation was originally reported for Last.fm-1K alone. Recency-based selection is retained as Essence's default mechanism on the strength of the Last.fm-1K and Amazon Books results; MovieLens-25M's mixed result is reported rather than smoothed over, and is consistent with this paper's broader finding that Essence's mechanisms behave differently where the evaluated collaborative baselines perform strongly, as on MovieLens-25M (Section 5.2).

5.9 Effect of User-Authored Review Embeddings (Amazon Books)

As a secondary experiment, we tested whether building user taste profiles from review text (first-person language describing a user's reaction to consumed items) rather than item metadata changes Essence's relative performance. Review coverage was complete for the 2,000-user Amazon Books subset.

Table 11: Amazon Books: metadata embeddings (Pass 1) vs. user-review embeddings (Pass 2). Pass 2 builds taste profiles from the user's own review text; candidates are scored using metadata embeddings. This secondary experiment uses $K = 3$ throughout and has not been re-run at Essence's validation-selected $K = 10$ (Section 3); it is reported here as a fixed-K diagnostic on the embedding-input question, not as a headline result.

Pass	System	Recall@10	LT-Recall@10	LT / Content
Pass 1 (metadata)	CF (ItemKNN)	0.0031	0.0137	—
	Content (Avg)	0.0173	0.0124	1.0×
	Essence ($K = 3$)	0.0221	0.0197	1.59×
Pass 2 (user review)	CF (ItemKNN)	0.0031	0.0137	—
	Content (Avg)	0.0027	0.0012	1.0×
	Essence ($K = 3$)	0.0041	0.0034	2.83×

Absolute recall degrades substantially in Pass 2. Two factors likely contribute. First, an input-distribution mismatch: taste profiles in Pass 2 are built from review-text embeddings, while candidate items are still scored using metadata embeddings. Although both are produced by the same encoder, review text and item metadata occupy different regions of that shared embedding space, and comparing across these regions is less discriminative than comparing inputs of the same type. Second, review text is a noisier input than structured metadata: a product review may describe shipping conditions or personal circumstances unrelated to the item's content, and an embedding built from off-topic text does not faithfully represent either the item or the user's response to it.

Essence's LT-Recall@10 stays above Content's in Pass 2, and the ratio rises from 1.59× to 2.83×. This rise should *not* be read as the K-means clustering mechanism coping well with degraded input. Both systems collapse in absolute terms, and the ratio grows chiefly because the denominator falls faster than the numerator: Content's LT-Recall@10 drops 90% (0.0124 → 0.0012) while Essence's drops 83% (0.0197 → 0.0034). A larger ratio between two collapsed quantities is not evidence of preserved capability, and at values this small the difference carries no statistical weight — we ran no significance test on this pair, and no comparison in this subsection is FDR-corrected. Such a reading would in any case sit against the clustering-null result of Section 5.10, which finds no measurable benefit from the clustering step once it is isolated from recency-weighted retrieval. We therefore treat Pass 2 as evidence that the embedding *input* matters — motivating the embedding-input-curation discussion in Section 7 — and not as evidence about the clustering mechanism.

5.10 Limitations, Stated as Findings

Three specific, quantified properties of Essence's behavior are reported here as findings.

The artist/author shortcut. On both datasets where this was tested, at $K = 3$ (not yet re-verified at the validation-selected K), roughly a fifth of Essence's top-10 recommendations shared an artist (Last.fm-1K, 20.6%) or author (Amazon Books, 19.8%) with an item in the active cluster — not the majority of recommendations, but a disproportionate share of the hits: excluding those same-artist/author candidates collapsed Recall@10 from 0.0119 to 0.0014 on Last.fm-1K and from 0.0221 to 0.0073 on Amazon Books (LT-Recall@10 from 0.0059 to 0.0005 and from 0.0197 to 0.0080, respectively), at $K = 3$. This analysis has not been re-run at Essence's validation-selected K ($K = 10$ for both datasets; Section 3); we report the $K = 3$ figures as the most recent verified numbers rather than assume they are unchanged. Some of Essence's apparent long-tail advantage is therefore closer to identity-based retrieval (more of what a known creator made) than to a genuinely novel content match, at least at $K = 3$, though the majority of recommendations are not same-artist/author.

Last.fm-1K's sample-size fragility. An earlier, small ($K \in \{2, \ldots, 5\}$) manual sweep on this dataset found Recall@10 rising sharply between $K = 3$ (0.0119) and $K = 4/K = 5$ (0.0239/0.0251), with K-means seed variance at fixed $K = 3$ (0.0122 ± 0.0011 across 10 seeds) far too small to explain that jump. This was an early signal that $K = 3$ was not a good fixed choice for this dataset specifically — one of the observations that motivated moving to the validation-selection procedure in Section 3 rather than a single fixed K across all three datasets. That procedure now supersedes the manual sweep, selecting $K = 10$ for Last.fm-1K on a held-out validation split. The underlying caveat about sample size is independent of K and still holds: long-tail comparisons throughout Last.fm-1K rest on very few eligible hits (as few as 2 of 92 users per method for the Essence-vs-Content comparison in Table 4), so Last.fm-1K's LT-Recall@10 comparisons should be read as directional rather than conclusive at any K. Last.fm-1K's headline numbers should be read with this fragility in mind.

The unresolved clustering-vs-recency-weighted question. The abstract raises, and does not fully resolve, how much of Essence's advantage where it exists comes from K-means clustering specifically versus recency-weighted retrieval alone. Two results bear on this directly, and both weaken — without reversing — at Essence's validation-selected K compared to an earlier, fixed-$K = 3$ version of this analysis. First, Essence's standing against Recency-Weighted on Recall@10 is no longer a clean loss on all three datasets: at $K = 3$, it lost significantly on all three; at the validation-selected K, it loses significantly only on MovieLens-25M ($d = -0.058$), and is not significantly different from Recency-Weighted on both Last.fm-1K ($d = -0.127$) and Amazon Books ($d = -0.009$) — the Amazon Books loss was significant at $K = 3$ ($d = -0.059$, $q = 0.021$) and is not at $K = 10$ ($q = 0.779$). Second, the same silhouette-stratification test as before — comparing Essence against Recency-Weighted within low/medium/high per-user clustering-quality tertiles, on the hypothesis that clustering should help most for genuinely multi-modal (high-silhouette) users — is rerun at the new K within the same pooled-family methodology (BH-FDR within the 52-comparison family combining the Last.fm-1K/Amazon Books baseline comparisons and the all-three-dataset stratification test, degenerate zero-variance comparisons excluded; Section 5.6). At $K = 3$, six of the seventeen clean Last.fm-1K/Amazon Books/MovieLens-25M stratification comparisons were significant losses for Essence, including the high-silhouette stratum on Last.fm-1K ($d = -0.315$, Recall@10) and MovieLens-25M ($d = -0.096$, Recall@10). At the validation-selected K (sixteen clean comparisons), only one remains significant: MovieLens-25M's high-silhouette stratum on LT-Recall@10 ($d = -0.092$). The Last.fm-1K high-silhouette loss and the MovieLens-25M high-silhouette Recall@10 loss are no longer significant ($d = -0.228$ and $d = -0.079$ respectively). The core qualitative conclusion is unchanged — no stratum, on any dataset or metric, at either K, shows Essence significantly *ahead* of Recency-Weighted — but the evidence that clustering *actively hurts* specific identifiable user groups is now substantially narrower than the $K = 3$ version of this analysis suggested: one significant loss, not five. If clustering were adding value specifically by separating genuinely distinct interests, the users most likely to show that value are still exactly the ones for whom it does not appear; we find no evidence that clustering helps, more so than evidence that it actively hurts. This does not mean clustering contributes nothing — the active cluster selection ablation (Section 5.8) shows recency-aware selection produces a numerical improvement on two of three datasets (Last.fm-1K, Amazon Books) on both metrics, with a mixed result on the third (MovieLens-25M: ahead on Recall@10, behind on LT-Recall@10); this ablation is descriptive and was not included in confirmatory inference — but it does mean the clustering step itself, isolated from recency weighting, is not empirically supported as the source of Essence's advantage where an advantage exists.

A separate, small-scale pilot (200 users, MovieLens-25M) testing whether adaptive per-channel weighting between two embedding channels could improve on Recency-Weighted found no directional win and remains unvalidated exploratory work, not reported here as a result.

6 Discussion

6.1 How the Apparent Conclusion Shifted Across the Evaluation

The headline conclusion this paper would have supported changed at every stage additional rigor or scope was added. Table 12 reconstructs this using only numbers already reported above (headline Recall@10, Table 3; significance results, Section 5.2; stratification results, Section 5.10) — no new comparisons are run for this table.

Each stage did not overturn the previous one so much as narrow what was actually being claimed: from "clustering beats collaborative filtering" to "recency-aware retrieval mostly matches or beats collaborative filtering, clustering's own contribution is domain-dependent, and neither the case for clustering nor the case against it survives an untuned K without narrowing further." This progression is the argument, developed in this section and Section 5.10, for reporting a mixed, domain-dependent result rather than the Stage-1 comparison alone.

6.2 A Plausible Mechanism: Why Popularity-Driven Discovery May Self-Reinforce

Collaborative filtering's reliance on cross-user co-occurrence is consistent with a well-documented feedback dynamic in the literature, not directly measured by this paper's static evaluation: items that already have more interactions can generate more training signal, potentially improving their representation in the model, increasing their likelihood of being recommended, and generating further interactions. Items with few interactions would not benefit from this loop, regardless of how well they might match any individual user's taste, because the signal needed to learn that match does not exist in sufficient quantity. This paper's evidence is consistent with, though not direct confirmation of, that dynamic: the non-personalized popularity baseline, which ranks purely by interaction count, records exactly zero long-tail recall on all three datasets (Table 4) — a structural consequence of the baseline's definition rather than an observation of the loop unfolding over time. Item-based CF records zero long-tail recall on the sparsest dataset, Last.fm-1K, under deterministic ranking; on the denser Amazon Books it surfaces some low-interaction items but pays a large cost to overall recall (0.0031, near the bottom of the field). Essence's retrieval step does not depend on this loop, since item representations are derived from content rather than interaction frequency: an item's likelihood of being recommended is a function of embedding similarity to the active cluster, independent of how many other users have interacted with it. MovieLens-25M is the case where this feedback dynamic runs at full strength in the other direction: CF's Recall@10 (0.0850) is the highest value recorded anywhere in this study, driven almost entirely by a small set of densely-rated popular titles, and Popularity itself ranks 3rd of 10 systems overall (0.0517) — yet even here, CF's LT-Recall@10 (0.0037) is far below its own overall Recall@10, confirming that strong aggregate performance from co-occurrence signal does not translate into long-tail recovery; it is domain conditions, not a fixed property of collaborative filtering, that determine how strong this feedback dynamic runs (Section 5.2).

Table 12: How the apparent headline conclusion shifts as evaluation stages are added, using only already-reported numbers. Each row's evidence is a subset of the final evaluation, not a separate experiment.

Stage	**Evidence at this stage**	**Apparent conclusion at this stage**
1. Naive baselines only	Amazon: Essence 0.0253 vs. CF 0.0031, Popularity 0.0021, Content 0.0173, Random 0.0001	Essence clearly beats collaborative filtering and popularity.
2. + recency baselines	Amazon: Last-Item 0.0311 and Recency-Weighted 0.0260 exceed Essence's 0.0253; Avg-Last-10 (0.0235) does not	Essence's apparent edge over CF was partly just recency, not clustering — plain recency retrieval beats Essence on 2 of 3 comparisons.
3. + paired bootstrap / FDR	Essence loses *significantly* to Last-Item ($d = -0.058$) on Amazon but is not significantly different from Recency-Weighted ($d = -0.009$); still significantly beats CF, Popularity, Content, and both neural baselines	Essence's real, statistically-supported value is beating collaborative/popularity/neural baselines, and it is not clearly behind simple recency-based retrieval either — only Last-Item remains a significant loss.
4. + third dataset (MovieLens)	Essence ranks 8th/10 on MovieLens (0.0053), behind CF (0.0850), Popularity (0.0517), ComiRec (0.0706), and MIND (0.0423), by effect sizes down to $d = -0.550$	Whether Essence's Amazon-style advantage holds is associated with how well the evaluated collaborative baselines perform in the domain, not shown to be predictive of it — not a fixed property of the architecture either way (Section 5.2).
5. + clustering-mechanism stratification	Essence never significantly beats Recency-Weighted in *any* silhouette stratum on any dataset; of sixteen clean stratum/metric comparisons, only one is a significant loss (MovieLens-25M's highest-silhouette stratum, LT-Recall@10, $d = -0.092$)	Even where Essence's relative standing is favorable, the K-means clustering step itself — isolated from recency-weighted retrieval — is not empirically supported as the source of that advantage; the evidence that it actively *hurts* specific user groups is narrow, not broad.
6. + validation-selected K (replacing a fixed $K = 3$)	Essence's own numbers rise on Last.fm-1K/Amazon Books at $K = 10$ (Amazon: 0.0221 $\rightarrow$ 0.0253; Last.fm-1K: 0.0119 $\rightarrow$ 0.0281), moving Essence from 5th/4th to 2nd/3rd of 10 on Recall@10; the Amazon Recency-Weighted loss ($d = -0.059$, significant at $K = 3$) becomes not significantly different ($d = -0.009$); the stratification family's significant-loss count drops from six of seventeen (at $K = 3$) to one of sixteen (at the new K)	Fixing $K = 3$ without validating it was itself understating Essence's standing on two of three datasets, and overstating how often clustering "actively hurts" specific user groups. Properly selecting K moves the story further toward domain-dependence and away from a uniform weakness relative to recency retrieval — MovieLens-25M's result, where CF signal is strong, is essentially unchanged.

6.3 Deployment Positioning: A Candidate-Generation Method, Not a Ranking Replacement — and Not a Universal One

The absolute recall values reported in Section 5 (approximately 0.005–0.03 under full-catalog evaluation) should be interpreted in light of how such a system would actually be deployed. Essence is not proposed as a replacement for a primary, CF-driven recommendation feed; CF baselines remain effective at surfacing broadly popular content efficiently, which is appropriate for a general homepage or default feed. On Amazon Books and Last.fm-1K, Essence is better understood as a specialized retrieval path: a candidate-generation method suited to a dedicated discovery surface (for example, a "more like this" or deep-catalog-exploration interface) where the explicit goal is surfacing items dissimilar from what population-level signals would otherwise recommend.

This positioning does not extend cleanly to every domain, and MovieLens-25M is the counterexample: where collaborative and popularity signal is strong, Essence trails not only CF and Popularity but both neural multi-interest baselines as well, by the largest effect sizes observed in this study (Section 5.2). A deployment decision may be informed by the performance of the evaluated collaborative baselines in the domain: whether Essence is a good candidate-generation method is associated with measurable properties of the domain, across the three domains evaluated here, rather than being a fixed property of peer-isolated, content-only personalization, although this study does not establish an independently measurable prospective predictor.

6.4 Privacy and Deployment Constraints

Essence may be well-suited to settings where collaborative filtering is not viable — privacy-regulated environments, on-device recommendation without server-side interaction logs, federated settings where data cannot leave the user's device, and cold-start contexts where a population-level interaction graph does not yet exist — given it consults no cross-user interaction data at any stage; this is a property of the architecture, not a claim of verified regulatory compliance in any specific jurisdiction. This positioning is consistent with a broader body of work surveying the deployment, training, and privacy constraints of recommenders that keep data local to the user rather than pooling it centrally [16]; Essence differs from most systems in that survey's scope in one respect: it requires no model training or update step at all, on-device or otherwise, since K-means is refit directly on a user's own history at inference time. This is a direct consequence of the architecture (Section 3) rather than an added feature, and follows from the same data-level distinction discussed in Section 2.2.

6.5 Robustness to Baseline Choice

The domain-dependence pattern discussed throughout this section rests partly on comparisons against a single collaborative-filtering baseline, CF-ItemKNN. Section 5.4 checks that this is not an artifact of that specific choice by implementing and conformance-testing BPR-MF, a pairwise-ranking matrix-factorization method with no architecture in common with ItemKNN. It replicates the pattern exactly: Essence beats it (raw $p < 0.05$, uncorrected) where collaborative signal is weak (Last.fm-1K, $d = +0.296$; Amazon Books, $d = +0.261$) and loses to it (raw $p < 0.05$, uncorrected) where collaborative signal is strong (MovieLens-25M, $d = -0.518$, ranking immediately behind CF-ItemKNN and ComiRec). This strengthens the domain-dependence claim against the objection that it depends on one particular, possibly-unrepresentative CF implementation, though it remains evidence from two collaborative methods, not a claim that the pattern holds for the family in general.

6.6 Broader Impact

A method that requires no cross-user interaction data reduces reliance on pooled cross-user interaction logs: personalization is possible without building or storing a pooled interaction graph across users, relevant in on-device, federated, or privacy-regulated deployment settings, as discussed above. The same property carries risk: retrieval driven entirely by a user's own centroid history can narrow what a user is shown toward their own past behavior (a filter-bubble effect, Section 7), and users with sparse or atypical interaction histories — for whom clustering has the least signal to work with — may receive degraded recommendation quality relative to users with richer histories, an equity concern this evaluation does not directly measure.

7 Limitations and Future Work

- **Scale.** Last.fm-1K evaluation uses 99 users. Amazon Books and MovieLens-25M each replicate on 2,000 users, but larger-scale evaluation across all three datasets would strengthen generalisability claims. In particular, the Last.fm-1K long-tail comparisons rest on few eligible long-tail hits (as few as 2 of 92 users per method) and are directional rather than statistically substantive (Section 5.10); the Amazon Books and MovieLens-25M results, at 2,000 users each, carry the statistical weight. A global-timestamp-cutoff split, tested as a robustness check and not adopted, strands 39% of Last.fm-1K's users (60/99 remain with data on both sides) and is uninformative at this scale, unlike Amazon Books, where the same check held up on a larger surviving sample.

- **Cross-user temporal leakage.** The chronological split (Section 4) is computed independently per user, with no shared global time cutoff. This preserves within-user chronology but does not enforce a global temporal cutoff; possible cross-user temporal leakage for population-based baselines (CF, Popularity) — an item's co-occurrence signal can include other users' interactions that occurred, in wall-clock time, after a given user's personal train/test cutoff — is therefore a limitation of this evaluation protocol, not eliminated by it.

- **K-selection anchoring.** K is now validation-selected per dataset (Section 3: $K = 10$ Last.fm-1K/Amazon Books, $K = 15$ MovieLens-25M), replacing a fixed $K = 3$. The one real caveat on the selection procedure itself: the stopping rule was applied moving forward from an already-established $K = 8$ checkpoint, not from $K = 2$. Applied retroactively to the $K \in \{2, \ldots, 5\}$ region, the rule would in fact have triggered earlier for Amazon Books and MovieLens-25M, since validation Recall@10 dips slightly across some of those closely-spaced early values before climbing again at higher K; this is disclosed as a scope decision in continuing an existing checkpoint forward, not a result the stopping rule itself produced from a clean start.

- **Active cluster selection.** The recency-based heuristic is simple by design. A learned selection mechanism could improve session-aware accuracy without requiring cross-user training.

- **Filter bubble risk.** Peer-isolation removes the serendipity that cross-user signals can provide. Whether repeated centroid queries cause semantic narrowing is an open empirical question.

- **Embedding input curation.** Section 5.9 demonstrates that noisy embedding inputs degrade performance. The quality of what enters the embedding determines the quality of what Essence can find. Automated curation of embedding input text (filtering off-topic or irrelevant content before encoding) is a direct practical extension.

- **Audio-based embeddings.** Current embeddings derive from item metadata text only. For music, incorporating audio-based representations (capturing sonic texture, timbre, rhythm, and harmonic structure directly from audio signal) would more precisely model the latent acoustic fingerprint underlying individual taste. This represents the primary planned extension of Essence: replacing or augmenting text embeddings with audio embeddings and measuring whether long-tail recall improves when the embedding captures what the music actually sounds like rather than what it is called.

- **Cross-space embedding alignment.** Pass 2 results suggest that aligning review-text and metadata input distributions within the shared embedding space (via contrastive training or a learned projection layer) could unlock the full signal of user-authored representations while maintaining retrieval coherence.

- **Hand-implemented, not pre-trained, multi-interest baselines.** MIND and ComiRec are evaluated here (Section 4) via from-scratch PyTorch implementations trained per dataset, since neither is available in standard recommendation libraries, rather than via pre-trained, production-tuned checkpoints. A well-tuned, larger-scale MIND or ComiRec deployment could plausibly outperform the versions evaluated here, particularly on MovieLens-25M, where both already outrank Essence substantially.

- **Rating threshold.** All Amazon and MovieLens ratings/reviews are treated as positive interactions regardless of rating value; this evaluates engagement/consumption prediction rather than preference prediction specifically. A sensitivity analysis using a positive-rating threshold (rating ≥ 4) was performed as a robustness check (below); it did not reverse the domain-dependence or clustering-null-result findings, though sample sizes drop 20–28% once users falling below the interaction floor under the stricter definition are excluded.

- **The domain-dependence mechanism is not fully explained.** "Collaborative/popularity signal strength" (Section 5.2) is measured post hoc via the baselines' own performance in each domain, not from an independent domain property decided in advance. A more principled, dataset-independent characterization of when peer-isolated personalization should be expected to help is left to future work.

- **Additional robustness checks.** Beyond the rating-threshold check above, multi-seed resampling (two additional user-sampling seeds, Amazon Books and MovieLens-25M) and validation-based ItemKNN hyperparameter tuning (neighbor count, shrinkage; test data never touched during the search) were each tested; neither reversed any reported finding's direction. CF-ItemKNN's Recall@10 on Amazon Books roughly doubles under validation-tuned shrinkage, but Essence still leads it by roughly 4×.

8 Conclusion

We presented Essence, a personal embedding cluster framework that generates recommendations by clustering a user's item history into K semantic centroids and retrieving items by similarity to the centroid closest to the user's recent activity, without consulting any cross-user interaction data. We evaluated Essence on three datasets spanning music, e-commerce, and film (Last.fm-1K, Amazon Books, MovieLens-25M) against nine baselines: a non-personalized popularity baseline, a real collaborative-filtering baseline, a content-based average-embedding baseline, three recency-based baselines that use the same embeddings as Essence but no clustering, and two hand-implemented multi-interest neural baselines (MIND, ComiRec).

Essence's performance relative to these baselines is domain-dependent: across the three evaluated datasets, Essence's rank among the ten evaluated systems is associated with the performance of the evaluated collaborative baselines in the domain (Section 5.2); establishing this as predictive would require additional datasets and an independently-defined domain property, not one measured post hoc via baseline performance. On Amazon Books and Last.fm-1K, where that signal is structurally weak, Essence sits in the upper half of the field, significantly beating collaborative filtering, popularity, content-averaging, and both neural baselines on Recall@10 or LT-Recall@10 in at least one dataset, and is not significantly different from most of the recency-based retrieval baselines that use the identical embeddings without any clustering step — all three on Last.fm-1K, two of three on Amazon Books, where only Last-Item remains a significant loss. On MovieLens-25M, where the evaluated collaborative baselines perform strongly, Essence ranks 8th of 10 systems, losing to collaborative filtering, popularity, and both neural baselines by the largest effect sizes observed anywhere in this study.

Whether peer-isolated, content-only personalization helps is associated with how well the evaluated collaborative baselines already perform in a given domain, across the three domains evaluated here; this association is not established as predictive of behavior on domains not yet tested. Recency-Weighted, a baseline sharing Essence's embeddings but no clustering, shows the identical pattern on Recall@10, so this domain-dependence is a property of the broader no-cross-user-pooling architecture family, not of K-means clustering specifically (Section 5.2). A targeted stratification test (Section 5.10) finds no evidence that the K-means clustering step, isolated from recency-weighted retrieval, is itself the source of Essence's advantage where an advantage exists. Essence is a data point on the domain conditions under which peer-isolated, content-only personalization is or isn't the right architectural choice, not a universally superior candidate generator.

An ablation (Section 5.8), now run on all three datasets at their validation-selected K, indicates that Essence's long-tail advantage, where present, depends at least partly on its recency-based active cluster selection mechanism specifically, not on K-means clustering alone: replacing recency-based selection with a static mean-embedding query degrades performance below the content baseline on Last.fm-1K and stays close to the content baseline (rather than clearly above it) on Amazon Books. On MovieLens-25M the result is mixed — recency-select leads on Recall@10 but mean-select leads on LT-Recall@10 — so this mechanism's benefit is not uniform across domains.

The long-tail definition used here (items with exactly one training-set interaction) was applied independently, without modification to the threshold or its derivation, to three datasets spanning different domains, scales, and interaction densities. The resulting singleton populations differ substantially in where they sit within the catalog — concentrated at the rare-popularity end on Last.fm-1K and Amazon Books (deciles 3–9), but more centrally on MovieLens-25M (deciles 2–5, Section 4) — a dataset-specific difference in decile geometry that the cold-start analysis (Section 5.7) shows matters for interpreting long-tail results correctly, not a failure of the definition itself.

Limitations of the present evaluation, beyond those above, are discussed in Section 7, including dataset scale, sensitivity to the choice of K, and the increased recall degradation observed when taste profiles are built from noisy, user-authored text (Section 5.9) rather than structured item metadata.

Future work should prioritize richer, disentangled item features and a more principled characterization of the domain property behind this pattern, validated against real user feedback rather than offline metrics alone; since clustering shows no measurable long-tail advantage over simple recency-weighting on any dataset tested, the more promising target for closing a long-tail gap is the embedding representation and retrieval mechanism, not the clustering step itself.

References

[1] Li, C., Liu, Z., Wu, M., Xu, Y., Zhao, H., Huang, P., Kang, G., Chen, Q., Li, W., & Lee, D. L. (2019). *Multi-Interest Network with Dynamic Routing for Recommendation at Tmall*. CIKM 2019.

[2] Cen, Y., et al. (2020). *Controllable Multi-Interest Framework for Recommendation*. KDD 2020.

[3] Sun, F., et al. (2019). *BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer*. CIKM 2019.

[4] He, X., et al. (2017). *Neural Collaborative Filtering*. WWW 2017.

[5] Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. IEEE Computer.

[6] Hidasi, B., et al. (2016). *Session-Based Recommendations with Recurrent Neural Networks*. ICLR 2016.

[7] Wang, W., et al. (2020). *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*. NeurIPS 2020.

[8] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP 2019.

[9] Celma, O. (2010). *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space*. Springer.

[10] Hou, Y., et al. (2024). *Bridging Language and Items for Retrieval and Recommendation*. arXiv:2403.03952.

[11] Krichene, W., & Rendle, S. (2020). *On Sampled Metrics for Item Recommendation*. KDD 2020.

[12] Ferrari Dacrema, M., Cremonesi, P., & Jannach, D. (2019). *Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches*. RecSys 2019.

[13] Ferrari Dacrema, M., Boglio, S., Cremonesi, P., & Jannach, D. (2021). *A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research*. ACM Transactions on Information Systems.

[14] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). *BPR: Bayesian Personalized Ranking from Implicit Feedback*. UAI 2009.

[15] Pazzani, M. J., & Billsus, D. (2007). *Content-Based Recommendation Systems*. The Adaptive Web, LNCS vol. 4321, pp. 325–341, Springer.

[16] Yin, H., Qu, L., Chen, T., Yuan, W., Zheng, R., Long, J., Xia, X., Shi, Y., & Zhang, C. (2024). *On-Device Recommender Systems: A Comprehensive Survey*. arXiv:2401.11441.

[17] Tan, Q., Zhang, J., Yao, J., Liu, N., Zhou, J., Yang, H., & Hu, X. (2021). *Sparse-Interest Network for Sequential Recommendation*. WSDM 2021.

[18] Zhang, S., Yang, L., Yao, D., Lu, Y., Feng, F., Zhao, Z., Chua, T., & Wu, F. (2022). *Re4: Learning to Re-contrast, Re-attend, Re-construct for Multi-interest Recommendation*. WWW 2022.

[19] Rendle, S. (2019). *Evaluation Metrics for Item Recommendation under Sampling*. arXiv:1912.02263.